Data Structures (ETCCDS202)

Unit–1 Assignment: Foundations & Algorithmic Analysis

Course Details :

Name of the School:	School of Engineering & Technology
Program (Class / Semester / Batch):	B. Tech (AI/ML / 2nd Semester / 2025-29)
Course Title:	Data Structures
Course Code:	ETCCDS202
Unit Title:	Foundations & Algorithmic Analysis
Student Name:	Anupam Chaudhary
Section:	A
Submission Date:	March 5th, 2026
Roll No.:	2501730151

Part A: Stack ADT

Definition :

A **Stack** is an Abstract Data Type (ADT) that follows the **Last-In First-Out (LIFO)** principle. It allows insertion (push) and deletion (pop) only at one end, called the top.

Implemented Operations

- push(x) → Insert element at top
- pop() → Remove element from top
- peek() → View top element without removing
- is_empty() → Check if stack is empty
- size() → Return number of elements

Complexity

- All operations run in **O(1)** time.
- Space complexity is **O(n)** for storing n elements.

Demonstration Task

We used the **Stack ADT to store mid-indices visited during recursive binary search**. This shows how a stack can record recursion traces, making the algorithm's flow transparent.

Part B: Factorial & Fibonacci

Factorial :

- Recursive definition:

$$n! = n \cdot (n - 1)!$$

- Complexity: **O(n)** time, **O(n)** space (recursion depth).

Fibonacci :

- **Naive recursion:**

$$F(n) = F(n - 1) + F(n - 2)$$

Complexity: **O(2^n)** time due to repeated calls.

- **Memoized recursion:** Stores results of subproblems. Complexity: **O(n)** time, **O(n)** space.

Observation

Memoization drastically reduces function calls compared to naive recursion, demonstrating the efficiency of **Dynamic Programming**.

Part C: Tower of Hanoi

Problem Statement :

Move n disks from source rod to destination rod using an auxiliary rod, following rules:

1. Only one disk moved at a time.
2. Larger disk cannot be placed on smaller disk.
3. All disks must be moved.

Recurrence Relation :

$$T(n) = 2 . T(n – 1) + 1$$

Complexity

- Time: **O(2^n)**
- Space: **O(n)** (recursion depth)

Example

For 3 disks → 7 moves.

Part D: Recursive Binary Search

Algorithm :

- Divide array into halves.
- Compare key with mid element.
- Recurse into left or right half.

Recurrence Relation :

$$T(n) = T(n/2) + O(1)$$

Complexity :

- Time: **O(log n)**
- Space: **O(log n)** (recursive stack)

Best & Worst Cases :

- Best case: Found at mid → **O(1)**
- Worst case: Not found → **O(log n)**

Part E: Paradigm Reflection

Paradigms Demonstrated :

- **Divide & Conquer** – Binary Search Splits the array into halves each step. Only one half is searched further.
    - Fast search, runs in O(log n) time.
    - Example: finding a word in a dictionary.
- **Dynamic Programming** – Memoized Fibonacci Stores results of subproblems to avoid repeating work.
    - Improves from O(2^n) to O(n) time.
    - Example: navigation apps storing shortest paths.
- **Recursion** – Tower of Hanoi & Factorial Defines problems in terms of smaller versions of themselves.
    - Elegant solutions but can be deep in calls.
    - Example: exploring folders inside folders.
- **Greedy** – Coin Change Problem Chooses the largest coin first at each step.
    - Works in many currency systems, but not always optimal.
    - Example: cashier giving change quickly with big notes

Real-World Applications :

- **Divide & Conquer:** Sorting large datasets (Merge Sort, Quick Sort).
- **Dynamic Programming:** Shortest path in navigation systems (Google Maps).
- **Recursion:** File system traversal, backtracking problems.
- **Greedy:** Activity selection, resource allocation.

Conclusion :

This assignment demonstrated:

- Implementation of Stack ADT and its practical use in tracing recursion.
- Comparison of naive vs optimized recursion in Fibonacci.
- Recursive solution for Tower of Hanoi with exponential complexity.
- Efficient searching using recursive binary search.
- Reflection on paradigms (Divide & Conquer, Dynamic Programming, Recursion, Greedy) with real-life examples.

Together, these tasks highlight the importance of **algorithmic paradigms** in designing efficient solutions to computational problems.

References :

- **Coin Change Problem :**
  GeeksforGeeks. *Coin Change – Count Ways to Make Sum (Dynamic Programming)*.
  https://www.geeksforgeeks.org/dsa/coin-change-dp-7
- **Tower of Hanoi Problem :**
  Tutorialspoint. *Data Structures and Algorithms – Tower of Hanoi*.
  https://www.tutorialspoint.com/data_structures_algorithms/tower_of_hanoi.htm